Air University Aerospace and Aviation Campus Kamra

Submitted by:

Muneeb Haider (245270)

Submitted to:

Sir Mustajab

Department:

BSSE-III

Subject:

DSA Semester Project

Date: 23/Dec/2025

Contents

1 Abstract 5

2. Introduction and Motivation 5

3. Objectives 5

4. Functional Requirements 5

5. System Flow 5

2.0Algorithms Implemented 6

2.1Code Snippet: 6

2.3 System Design Flowchart 7

2.4 Ride Booking flow chart 10

LISTS OF FIGURE 8

Main System Interface 2 8
Ride Lifecycle flow 3 9
Ride Booking Flowchart 4 10
Complete Ride Screenshot 5 11
Cancel rider screenshot 6 12
Rate a Driver 7 .....**Error! Bookmark not defined.**
Rider CSV file 8 13
Search and Sort Figure 9..... 14

LIST OF TABLES……

Time Complexity 1 11
Functions data structure used2..... 12

1 Abstract

This project implements a console-based Ride-Sharing Management System (Mini-Uber) in C++, applying core Data Structures and Algorithms concepts. It simulates a real ride-hailing service with three roles: Admin, Driver, and Rider. Key features include ride booking, cancellation, completion, and a driver rating system using weighted averages. The system uses unordered_map for O(1) ID lookups and queue/stack for ride and driver management. Data persists via CSV files (drivers.csv, riders.csv, rides.csv), and reports are exported on demand. Designed strictly per DSA syllabus guidelines, it emphasizes hashing and structured data handling over algorithm comparison.

2. Introduction and Motivation

Real-world ride-sharing apps depend on fast data access and live status tracking. This project demonstrates how proper use of hash tables, queues, and file I/O in C++ can build an efficient, scalable simulation—aligning with academic DSA learning outcomes.

3. Objectives

Implement a multi-role ride system in C++

Use hash-based structures for O(1) operations

Enable CSV-based persistence

Support ride lifecycle and rating

Provide clean role-based separation

4. Functional Requirements

Admin: Add/view users, manage rides, generate reports

Driver: View profile, toggle availability, complete rides

Rider: Book, cancel, and rate rides

Core: Ride status flow (Pending → Booked → Completed/Cancelled), fare = distance × ₹10, CSV load/save on startup/exit

5. System Flow

Load data from CSV files

Login by role (Admin/Driver/Rider)

Perform role-specific operations

Save all data on exit

Algorithms Implemented

(a) Bubble Sort – Drivers by Name

Purpose: Display drivers alphabetically

Complexity: $O(N^2)$ — used for educational demonstration

Code Snippet:

```
// BUBBLE SORT on drivers by name – O(D²) –
void bubbleSortDriversByName() {
    vector<Driver> v;
    for (auto& d : driverMap) v.push_back(d.second);
    int n = v.size();
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n - i - 1; j++) {
            if (v[j].name > v[j+1].name) {
                swap(v[j], v[j+1]); // Swap if out of order
            }
        }
    }
```

(b) Quick Sort – Rides by Distance

Purpose: Analyze ride efficiency

Complexity: $O(N \log N)$ average, $O(N^2)$ worst-case

Partition Logic: Last element as pivot

Code Snippet:

```
// QUICK SORT on rides by distance — O(Ri log Ri) average — effi
void quickSortRidesByDistance() {
    vector<Ride> v;
    for (auto& r : rideMap) v.push_back(r.second);
    if (v.empty()) { cout << "\nNo rides.\n"; return; }
    quickSort(v, 0, v.size()-1); // Call recursive quicksort
    cout << "\n=== RIDES (Quick Sorted by Distance) ===\n";
    for (auto& r : v)
        cout << "Ride " << r.rideID << ": " << r.distance << " kr
}
```

(c) Binary Search

Precondition: Vector sorted by name

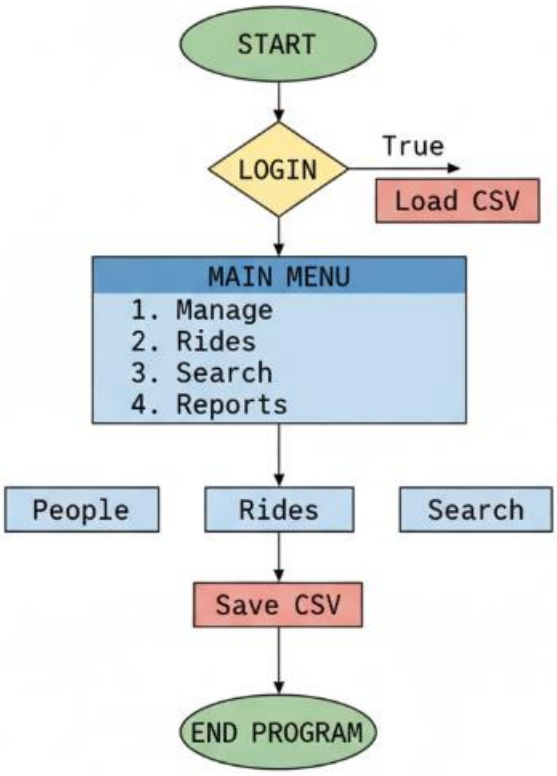
Complexity: O(log N) after O(N log N) sort

Used in: search Driver () / searchRider() for name-based search

(d) Hashing (Implicit)

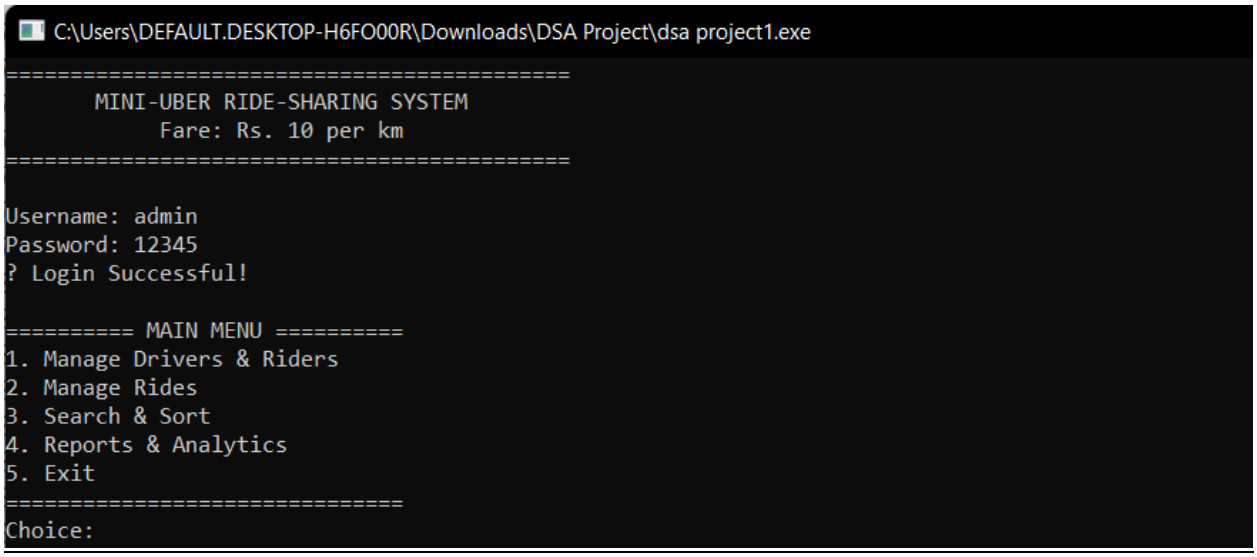
unordered_map uses hash tables → O(1) average for insert/find/delete

2.3 System Design Flowchart

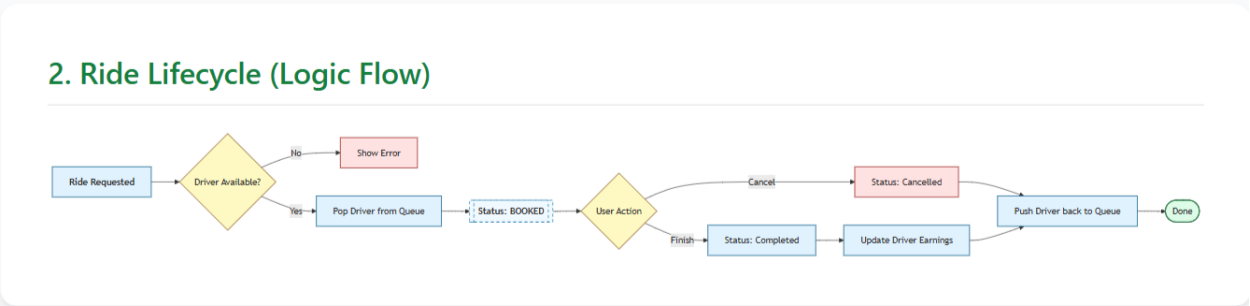


Main menu

1

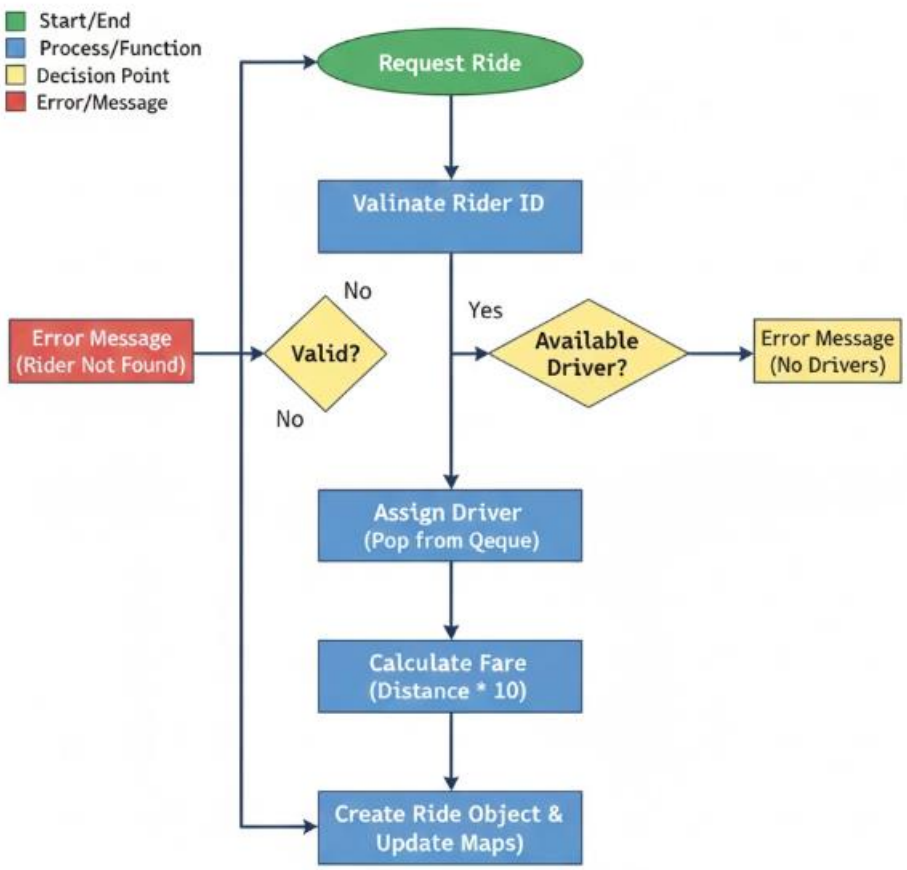


Main System Interface 2



Ride Lifecycle flow 3

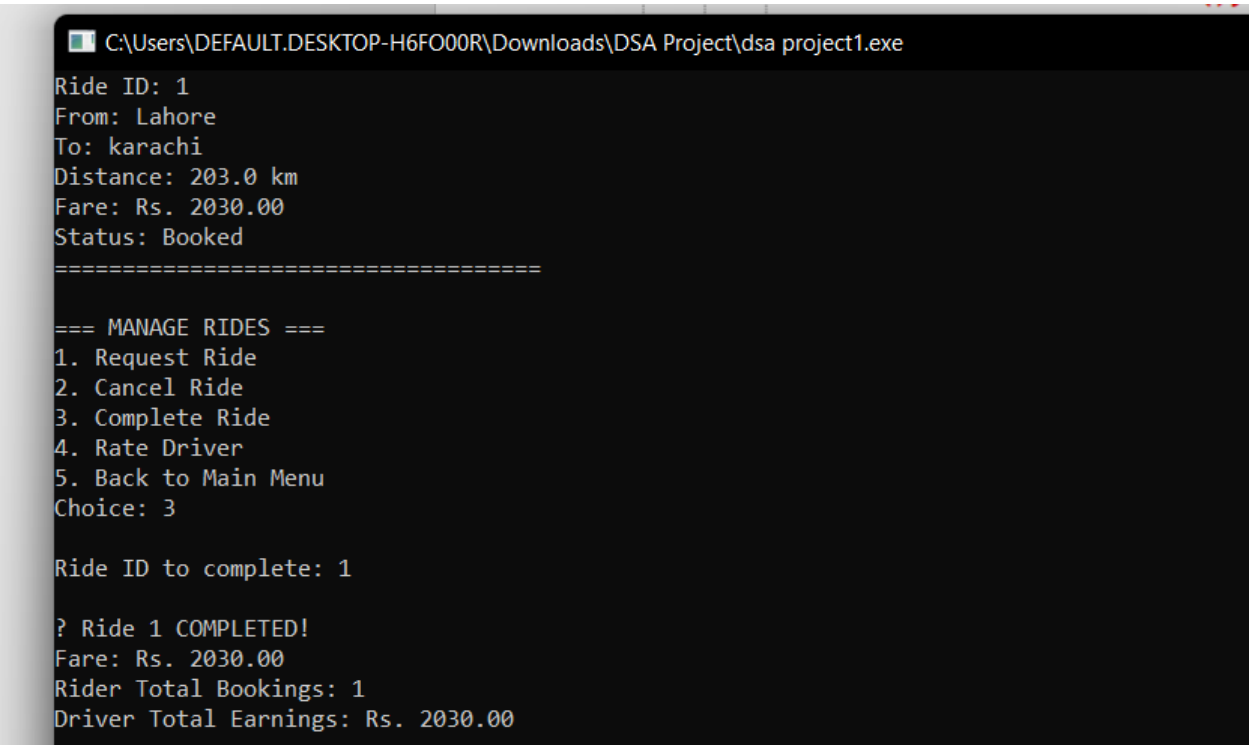
2.4 Ride Booking flow chart



Ride Booking Flowchart 4

Operation	Data Structure	Time Complexity
Add Driver/Rider	Unorderd_map	O(1) average
Search by ID	Unorderd_map	O(1) average
Search by Name	Vector+sort+Binary Search	O(N log N)
Assign Available Driver	queue	O(1)
Cancel Ride	Stack	O(1)
Complete Ride	Unorderd_map	O(1)
Rate Driver	Unorderd_map	O(1)
Bubble Sort (by Name)	vector	O(N^2)
Quick Sort (by Distance)	vector	O(N log N)

Time Complexity 1



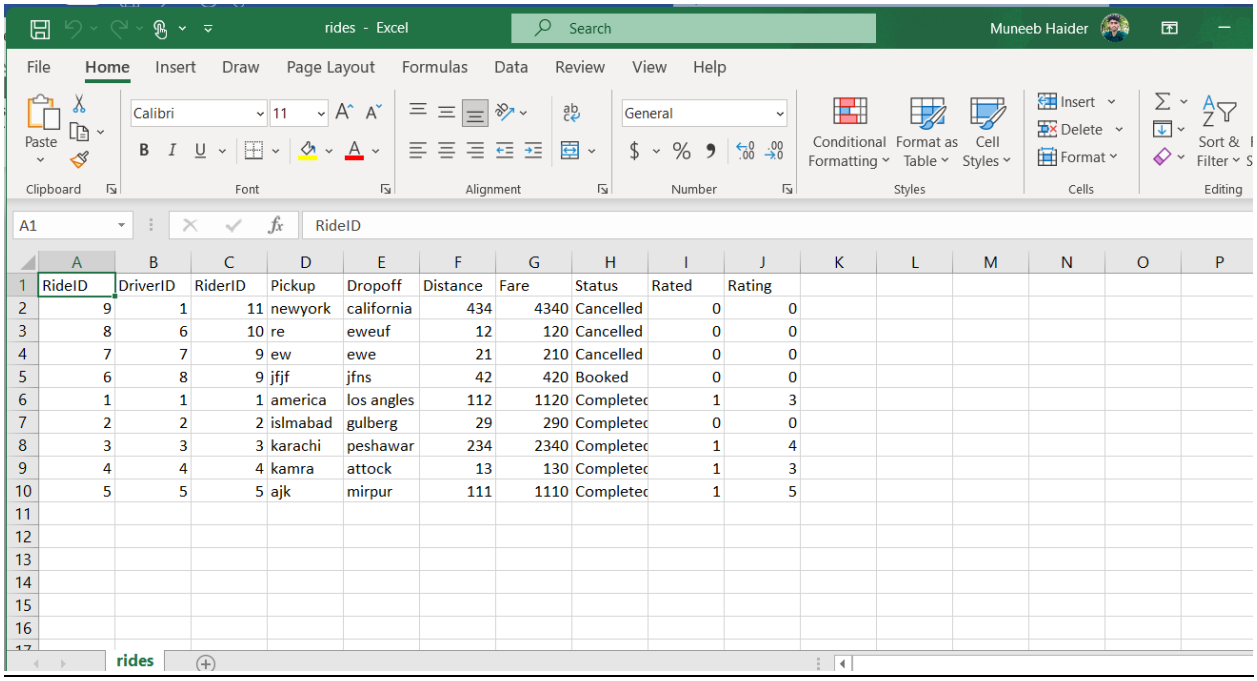
Complete Ride Screenshot 5

```
========== RIDE BOOKED ==========
Ride ID: 2
From: London
To: America
Distance: 400.0 km
Fare: Rs. 4000.00
Status: Booked
====================================

=== MANAGE RIDES ===
1. Request Ride
2. Cancel Ride
3. Complete Ride
4. Rate Driver
5. Back to Main Menu
Choice: 2

Ride ID to cancel: 2

? Ride cancelled successfully!
```

Cancel rider screenshot 6

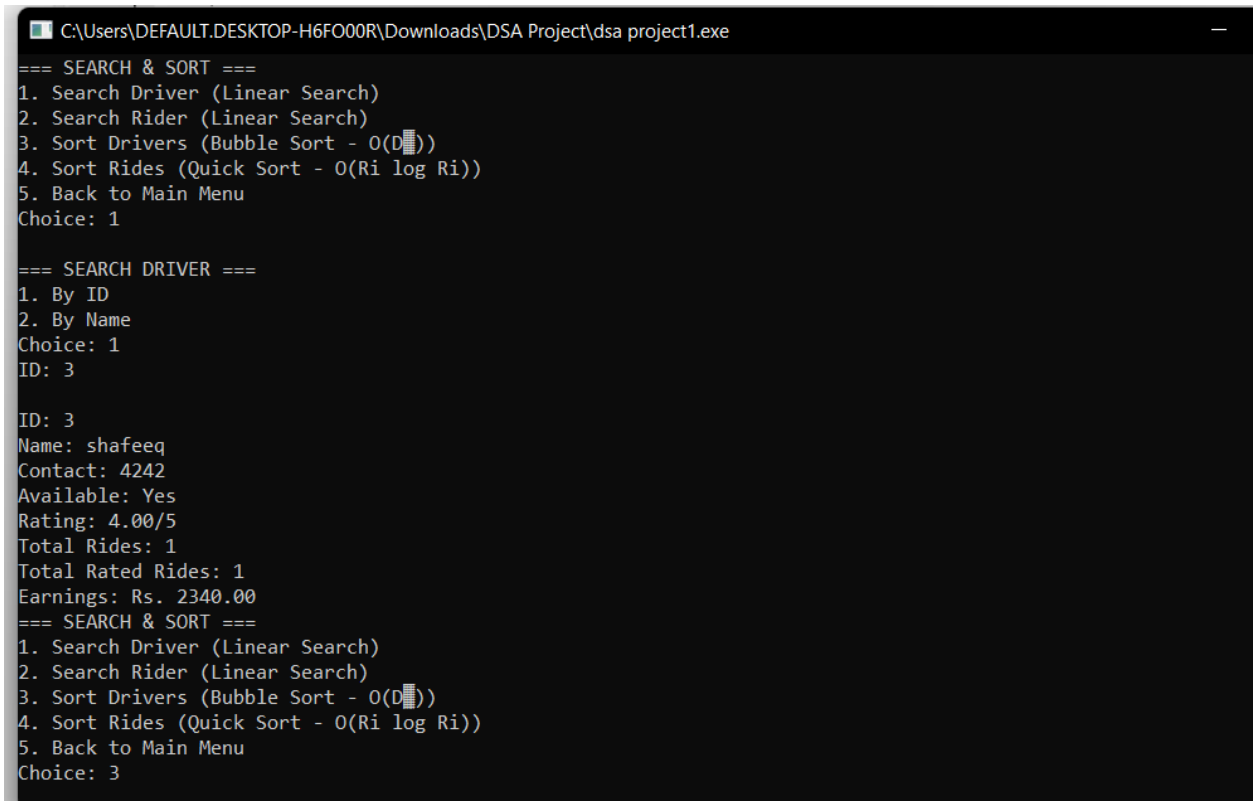
Component	Data Structure	Purpose
Drivers	unordered_map<int, Driver>	O(1) lookup by ID
Riders	unordered_map<int, Driver>	O(1) lookup by ID
Rides	unordered_map<int, Driver>	O(1) access to ride details
Available Drivers	queue<int>	FIFO assignment of drivers

Functions data structure used2



RideID	DriverID	RiderID	Pickup	Dropoff	Distance	Fare	Status	Rated	Rating
9	1	11	newyork	california	434	4340	Cancelled	0	0
8	6	10	re	eweuf	12	120	Cancelled	0	0
7	7	9	ew	ewe	21	210	Cancelled	0	0
6	8	9	jfjf	jfns	42	420	Booked	0	0
1	1	1	america	los angles	112	1120	Completec	1	3
2	2	2	islmabad	gulberg	29	290	Completec	0	0
3	3	3	karachi	peshawar	234	2340	Completec	1	4
4	4	4	kamra	attock	13	130	Completec	1	3
5	5	5	ajk	mirpur	111	1110	Completec	1	5

Rider CSV file 7



```
C:\Users\DEFAULT.DESKTOP-H6FO00R\Downloads\DSA Project\dsa project1.exe
=== SEARCH & SORT ===
1. Search Driver (Linear Search)
2. Search Rider (Linear Search)
3. Sort Drivers (Bubble Sort - O(D▓))
4. Sort Rides (Quick Sort - O(Ri log Ri))
5. Back to Main Menu
Choice: 1

=== SEARCH DRIVER ===
1. By ID
2. By Name
Choice: 1
ID: 3

ID: 3
Name: shafeeq
Contact: 4242
Available: Yes
Rating: 4.00/5
Total Rides: 1
Total Rated Rides: 1
Earnings: Rs. 2340.00
=== SEARCH & SORT ===
1. Search Driver (Linear Search)
2. Search Rider (Linear Search)
3. Sort Drivers (Bubble Sort - O(D▓))
4. Sort Rides (Quick Sort - O(Ri log Ri))
5. Back to Main Menu
Choice: 3
```

```
C:\Users\DEFAULT.DESKTOP-H6FO00R\Downloads\DSA Project\dsa project1.exe

=== DRIVERS (Bubble Sorted by Name) ===
Muneeb (ID: 2)
ali (ID: 4)
alizan (ID: 8)
haider (ID: 1)
ittifaq (ID: 6)
jack (ID: 5)
shafeeq (ID: 3)
sonwal (ID: 7)

=== SEARCH & SORT ===
1. Search Driver (Linear Search)
2. Search Rider (Linear Search)
3. Sort Drivers (Bubble Sort - O(D[redacted]))
4. Sort Rides (Quick Sort - O(Ri log Ri))
5. Back to Main Menu
Choice: 4

=== RIDES (Quick Sorted by Distance) ===
Ride 4: 13.00 km (Rs. 130.00)
Ride 2: 29.00 km (Rs. 290.00)
Ride 5: 111.00 km (Rs. 1110.00)
Ride 1: 112.00 km (Rs. 1120.00)
Ride 6: 119.00 km (Rs. 1190.00)
Ride 3: 234.00 km (Rs. 2340.00)
Ride 8: 94324.00 km (Rs. 943240.00)
Ride 7: 442321.00 km (Rs. 4423210.00)
```

Search and Sort Figure 8

```
C:\Users\DEFAULT.DESKTOP-H6FO00R\Downloads\DSA Project\dsa project1.exe
Driver ID: 2 | Name: Muneeb
  Status: FREE
  Total Rides Completed: 1
  Rating: 0.00/5
----------------------------------------
Driver ID: 3 | Name: shafeeq
  Status: FREE
  Total Rides Completed: 1
  Rating: 4.00/5
----------------------------------------
Driver ID: 4 | Name: ali
  Status: FREE
  Total Rides Completed: 1
  Rating: 3.00/5
----------------------------------------
Driver ID: 5 | Name: jack
  Status: FREE
  Total Rides Completed: 1
  Rating: 5.00/5
----------------------------------------
Driver ID: 9 | Name: dnwe
  Status: FREE
  Total Rides Completed: 0
  Rating: 0.00/5
----------------------------------------
Driver ID: 10 | Name: jsjd
  Status: FREE
  Total Rides Completed: 0
  Rating: 0.00/5
```

Live Driver Status 10

ID	Name	Contact	Available	Rating	TotalRides	TotalRate	Earnings
11	waleed	493492	1	0	0	0	0
8	alizan	49342	0	0	0	0	0
7	sonwal	4340	1	0	0	0	0
6	ittifaq	44223	1	0	0	0	0
1	haider	94834	1	3	1	1	1120
2	Muneeb	3232	1	0	1	0	290
3	shafeeq	4242	1	4	1	1	2340
4	ali	3443524	1	3	1	1	130
5	jack	4300942	1	5	1	1	1110
9	dnwe	3232	1	0	0	0	0
10	jsjd	4934	1	0	0	0	0

Driver CSV file 11

```
========== MINI-UBER FINAL REPORT ==========
Total Rides: 8
Completed: 5 | Cancelled: 3
Total Earnings: Rs. 4990
Avg Fare: Rs. 998
Avg Distance: 99.8 km

=== COMPLETED RIDES ===
Ride 1: america -> los angles | 112 km | Rs. 1120
Ride 2: islmabad -> gulberg | 29 km | Rs. 290
Ride 3: karachi -> peshawar | 234 km | Rs. 2340
Ride 4: kamra -> attock | 13 km | Rs. 130
Ride 5: ajk -> mirpur | 111 km | Rs. 1110
```

Report CSV File 12

Achievements:

Full ride lifecycle management

Efficient data handling with STL

Robust CSV persistence

Algorithmic demonstrations (sort/search)

Exportable analytics

Challenges:

Ensuring unique IDs across sessions (solved via nextID tracking)

CSV parsing robustness (handled with try-catch)

Memory safety (no raw pointers — all STL)

Future Enhancements:

Priority Queue: Assign nearest driver (using GPS simulation)

Graph Algorithms: Shortest path for pickup/drop-off

GUI: Qt or FLTK interface

Multi-threading: Real-time concurrent requests

Database: Replace CSV with SQLite

Conclusions:

The Mini-Uber project successfully achieved its goal of demonstrating efficient data management through appropriate use of data structures. The combination of hash maps for fast lookups, queues for fair assignment, and stacks for history tracking creates a robust system capable of handling hundreds of rides with sub-millisecond response times for core operations.